面向Ascend 950的Ascend C SIMD/SIMT编程实践

分享嘉宾：张浩

2026.4.18　中国·北京

content

目录

01 Ascend 950硬件架构介绍

02 Ascend C SIMT编程实践

03 Ascend C SIMD+SIMT混合编程实践

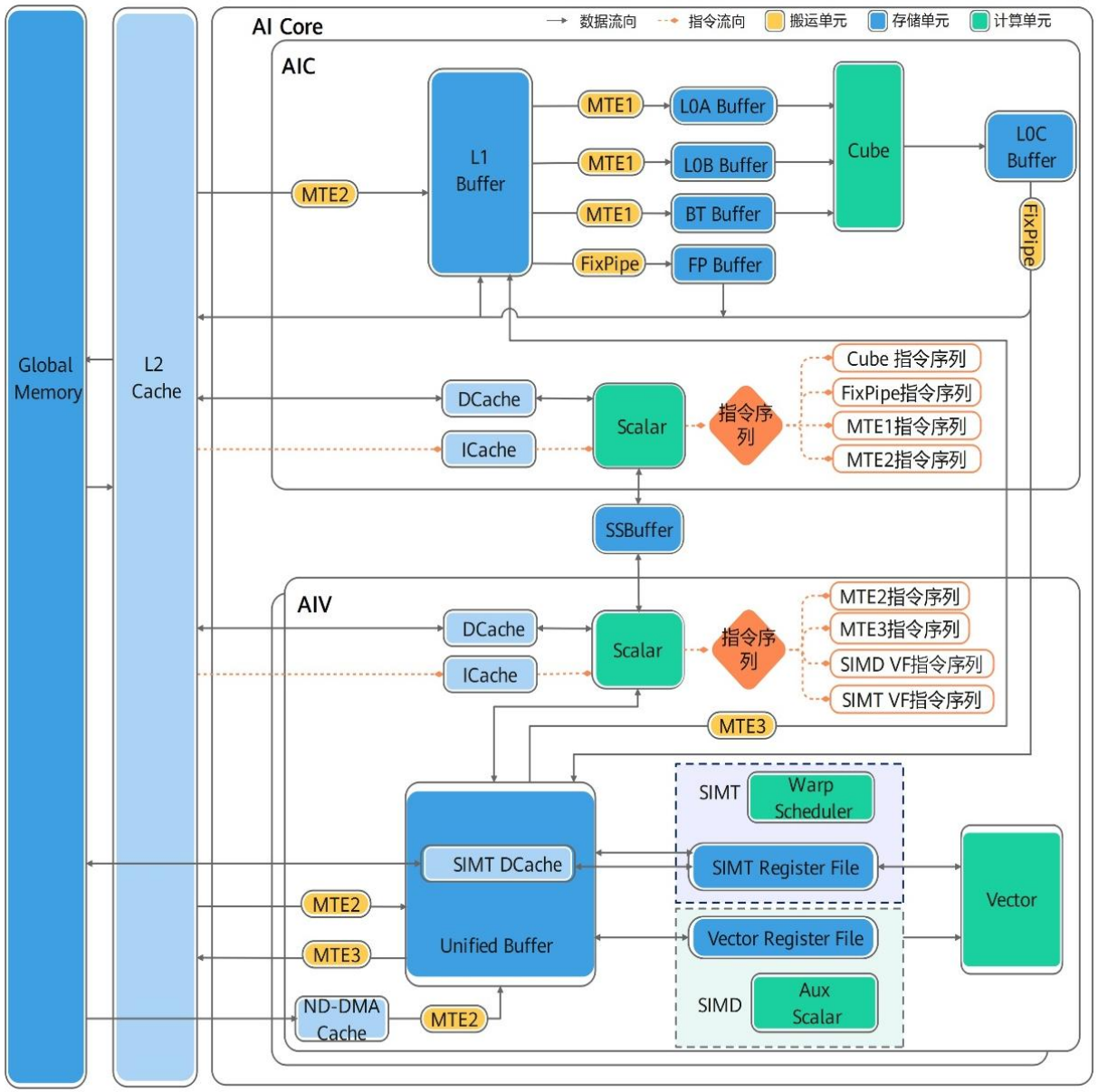
04 Ascend C SIMT算子优化实践

05 Ascend C开源仓介绍

Ascend 950硬件架构介绍

01

Ascend 950硬件架构介绍



相比 A2/A3 Vector Core，A5新增SIMT单元

SIMT:

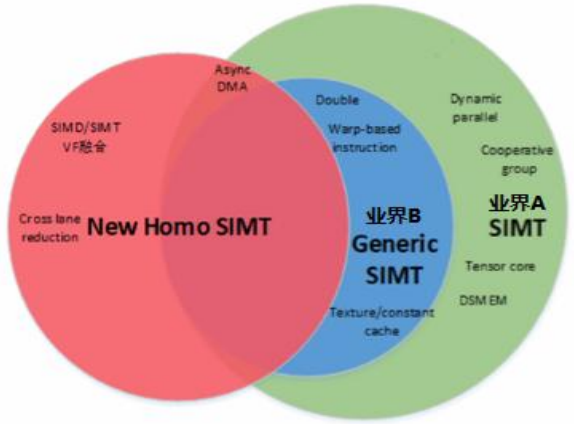
- 支持直接读写GM或UB (UB当做Shared Memory)
- 一个SIMT Vector Function 运行一个 thread block，同一时刻一个AIV只能执行一个SIMT Vector Function
- 每个AIV有**4个**Warp Scheduler, scheduler id = warp id % 4
- 最大支持**128KB** Data Cache，Data Cache直接复用UB作为Cacheline
- 不支持Independent Thread Scheduling、不支持多Block同时调度;

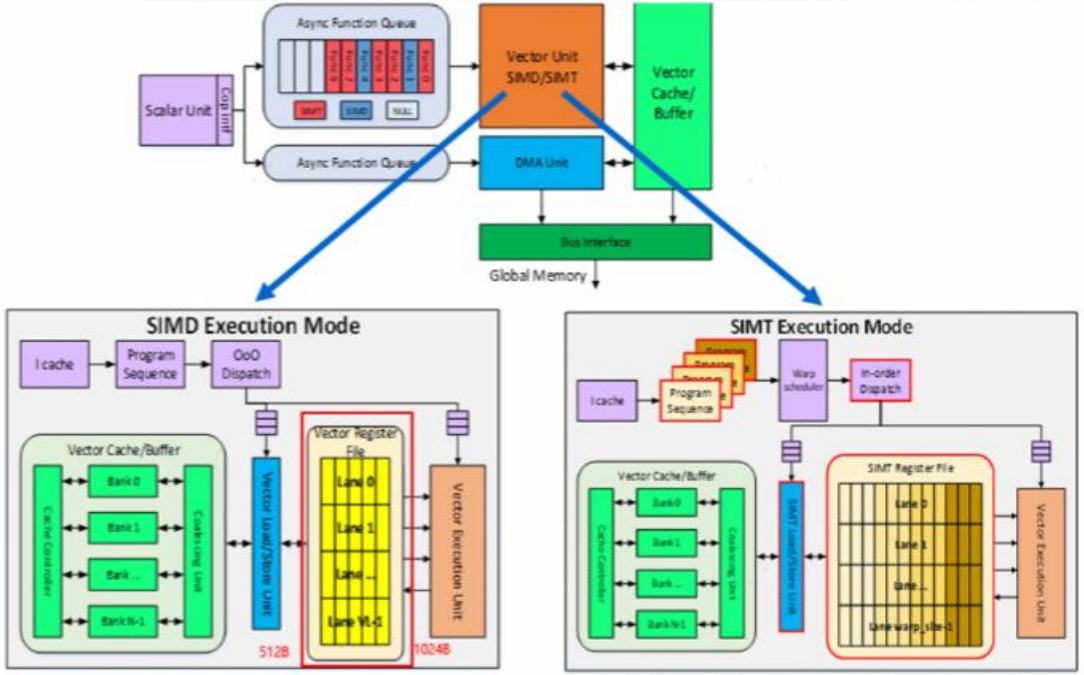
SIMD:

- SIMD Register大小为**256B**;
- 需要显式Load/Store将数据从UB搬运到Reg；从SIMD Reg访问UB理论带宽上限**512B/cycle**;
- 不支持直接从GM搬运数据到SIMD Reg
- **额外新增Aux Scalar，主要用于处理SIMD VF内部标量计算，能力弱于AIV Main Scalar**

Ascend 950硬件架构介绍

- Scalar 计算单元将Vector Function发射到Vector Function Queue中；
- 同时支持两种模式：SIMD 、SIMT
- 工作模式以Vector Function为粒度进行切换；
- UB(Shared Memory）是SIMD和SIMT共享的；执行上下文(UB Dcache）在VF切换时会保留；



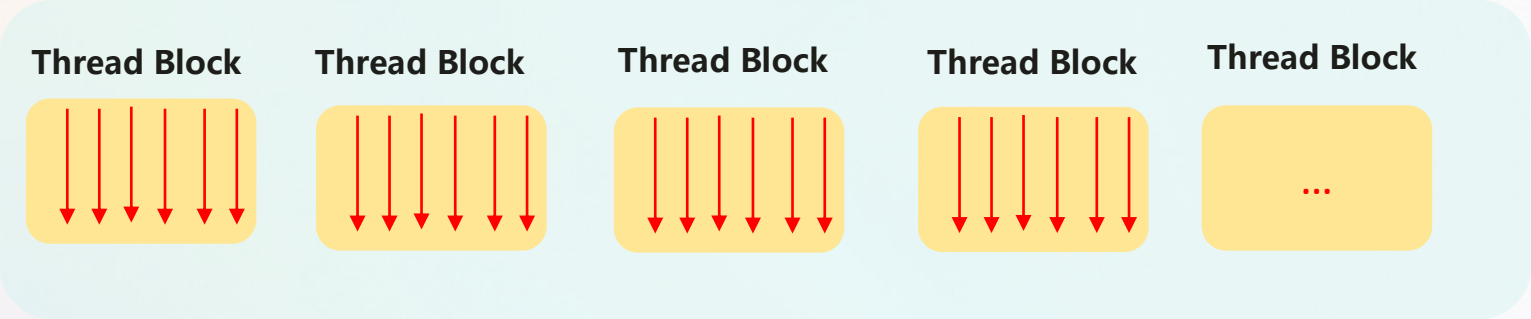


Ascend C SIMT编程实践

02

Ascend C SIMT编程实践

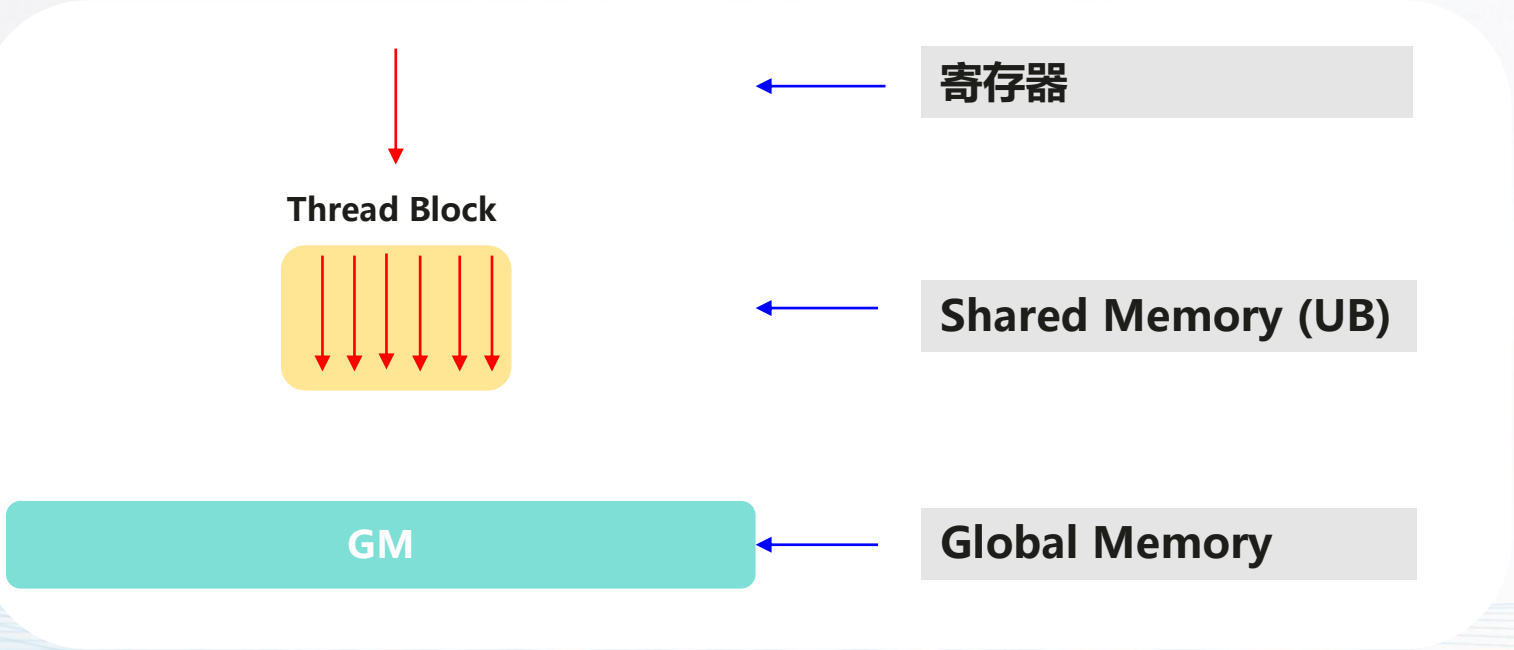
Ascend 950 SIMT 线程架构



核心概念:

- block个数， gridDim (dim3)
- 一个block的线程个数, blockDim(dim3)
- 一次统一调度的线程组， warp

Ascend950 内存模型



- 一个SIMT Vector Function 运行一个 thread block，同一时刻一个AIV只能执行一个SIMT Vector Function
- 每个AIV有**4个**Warp Scheduler, scheduler id = warp id % 4
- 一个Thread Block 最多支持**2048**个线程；

线程个数	寄存器个数
2048	16
1024	32
512	64

Ascend C SIMT编程实践

类别	ASC 编程API	业界SIMT编程接口	说明
内置变量	gridDim、blockIdx、blockDim、threadIdx、warpSize	gridDim、blockIdx、blockDim、threadIdx、warpSize	
内置函数/内存修饰符	__global__、__aicore__ 、__host__ __ubuf__、	__global__ 、__device__、__host__ __shared__	
核函数定义	__global__ __launch_bounds(maxThreadsPerBlock) void myKernel(float* input, float*output, int size);	__global__ __launch_bounds(maxThreadsPerBlock, minBlocks) void myKernel (float* input, float*output, int size);	

核函数调用	dim3 numBlocks; //GridDim;
	dim3 threadsPerBlock; //BlockDim
	numBlocks 和 threadsPerBlock 同时支持int或dim3 类型;
	kernel_name<<<numBlocks, threadsPerBlock, dynUBufSize, stream>>>(args...)
	
	dynUBufSize : 动态申请内存空间总大小
		dim3 numBlocks; //GridDim;
		dim3 threadsPerBlock; //BlockDim
		numBlocks 和 threadsPerBlock 同时支持int或dim3 类型;
		
		kernel_name<<<numBlocks, threadsPerBlock, shareMemSize, stream>>>(arg s...)	

Memory Fence	__aicore__ inline void asc_threadfence()、asc_threadfence_block()	__threadfence() 、__threadfence_block()、__threadfence_system	
同步函数	__aicore__ inline void asc_syncthreads()	__syncthreads()、__syncthreads_count、__syncthreads_and、 __syncwarp	
Warp Vote	int32_t asc_all(int32_t predicate) 、asc_any(int32_t predicate)、 asc_activemask()	int __all(int predicate); // 9.0之前 int __all_sync(unsigned mask, int predicate); // 9.0新增	支持9.0之前不带mask版本

Warp Shuffle	__aicore__ inline int32_t asc_shfl(int32_t var, int32_t srcLane, int32_t width = warpSize)、asc_shfl_up、asc_shfl_down、asc_shfl_xor	int __shfl(T var, int srcLane, int width=warpSize); // 9.0之前
		int __shfl_sync(unsigned mask, int var, int srcLane, int width=warpSize);
		__shf_up_sync、__shfl_down_sync、__shfl_xor_sync // 9.0 版本新增	

Warp Reduce	__aicore__ inline unsigned asc_reduce_add(unsigned val) 、asc_reduce_max、asc_reduce_min	unsigned __reduce_add_sync(unsigned mask, unsigned value) 、 _reduce_min_sync、_reduce_max_sync // 11.0 版本新增	
原子函数	__aicore__ inline int_t asc_atomic_add(int *address, int val) 、asc_atomic_sub、asc_atomic_exch等_	int atomicAdd(int* address, int val)、atomicSub、atomicExch等	
Math函数	__aicore__ inline float asinf(float x)、__aicore__ inline float atanf(float x) 等	asinf(x) 、atanf(x) 等	因基础函数跟C标准库中math基本一致，因此跟随C Math函数命名保持一致
Launch Bound	__launch_bound(maxThreadsPerBlock)	__launch_bound(maxThreadsPerBlock, minBlocks)， minBlocks可选	

Ascend C SIMT编程实践

以典型的Reduce算子为例

```
__global__ void reduceKernel(float* input, float* output, int N) {
    extern __ubuf__ float sdata[]; // Shared memory for reduction
    int tid = threadIdx.x;
    int i = blockIdx.x * blockDim.x + threadIdx.x;

    // Load data into shared memory
    if (i < N) {
        sdata[tid] = input[i];
    } else {
        sdata[tid] = 0.0f; // Handle out-of-bounds access
    }
    asc_syncthreads();

    // Perform reduction in shared memory
    for (unsigned int s = blockDim.x / 2; s > 0; s >>= 1) {
        if (tid < s) {
            sdata[tid] += sdata[tid + s];
        }
        asc_syncthreads();
    }

    // Write result to global memory (only by thread 0 of each block)
    if (tid == 0) {
        output[blockIdx.x] = sdata[0];
    }
}

int main(int argc, char* argv[])
{
    ...
    // Launch the ASC Kernel (it also support dim3 for threadsPerBlock and blocksPerGrid)
   int threadsPerBlock = 256;
   int blocksPerGrid = (size + threadsPerBlock - 1) / threadsPerBlock;
   reduceKernel<<<blocksPerGrid, threadsPerBlock , dynUBufSize>>>(d_idata, d_odata, size);
   ...
  aclFinalize();
  return 0;
}
```

Ascend C SIMD+SIMT混合编程实践

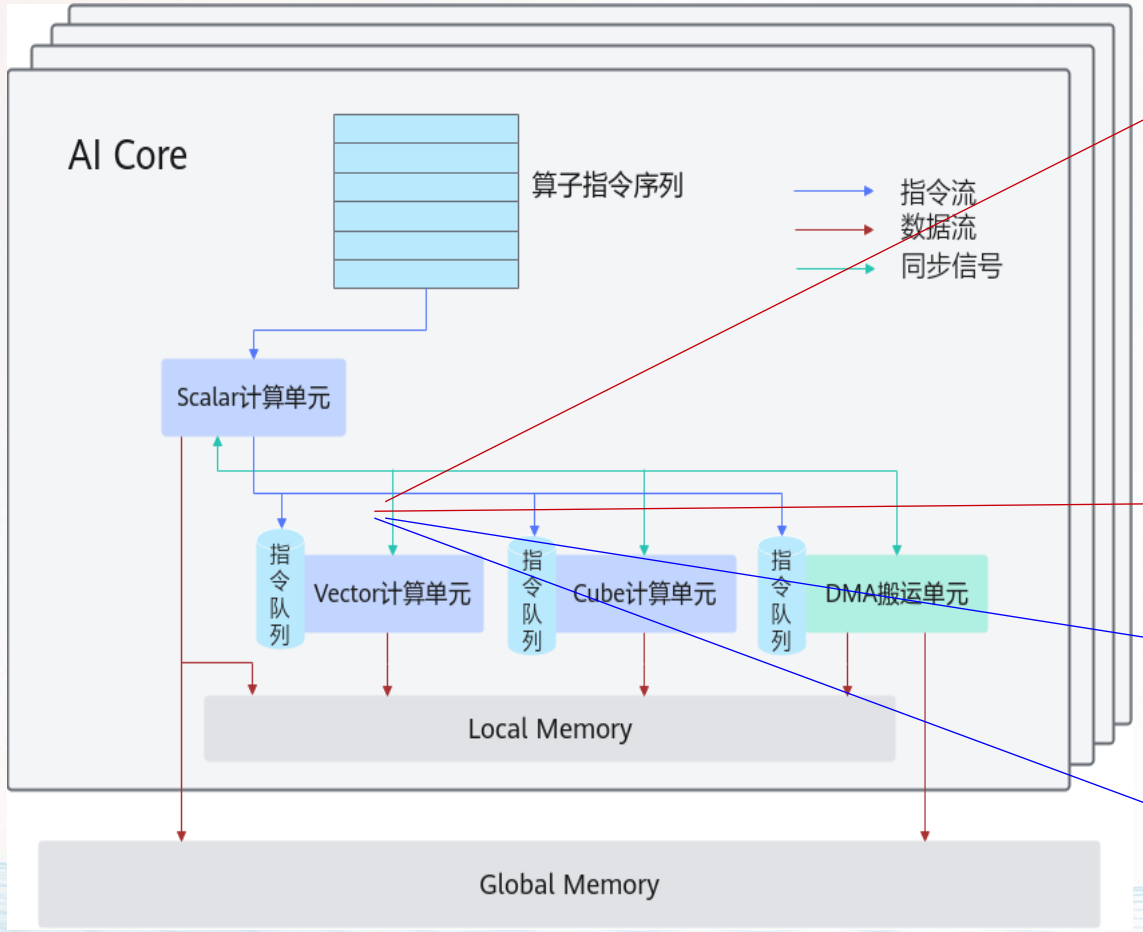
03

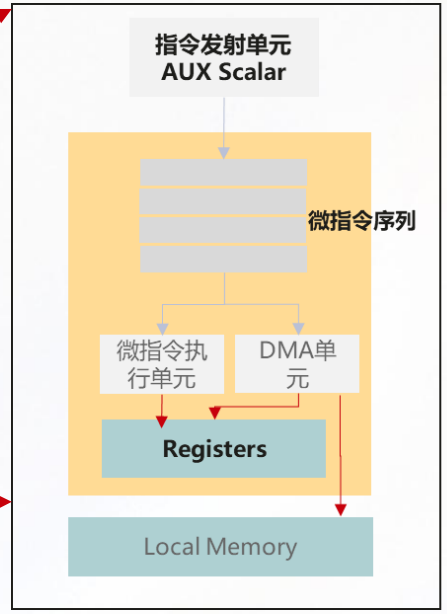
Ascend C SIMD+SIMT混合编程实践

核心概念：

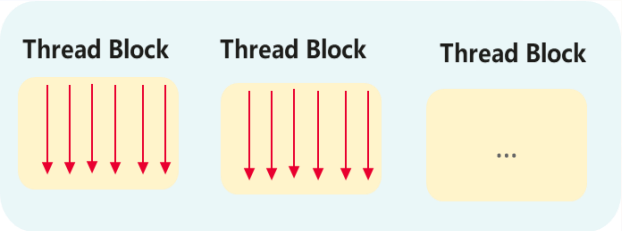
- 一个Block可以调用SIMD VF（Vector Function）也可以调用SIMT VF
- 在调用SIMT VF时，需要指定启动多少个线程；

Ascend950 SIMD/SIMT 架构





SIMD Vector计算单元



SIMT Vector计算单元

Ascend C SIMD+SIMT混合编程实践

类别	ASC 混合编程SIMT API	业界SIMT编程接口	说明
内置变量	① SIMT VF: gridDim、blockIdx、blockDim、threadIdx、warpSize、subBlockIdx(仅aic/aiv mix场景需要) ① SIMD VF: block_num、block_idx、sub_block_idx	① gridDim、blockDim、threadIdx、warpSize、blockIdx	基于A2兼容的SIMD 提供GetBlockNum() 、block_idx、block_num
内置函数/内存修饰符	① __global__、__simt_vf__、__aicore__ 、__host__、__simd_vf__、__simt_callee__ 、__simd_callee__ ② __ubuf__、__gm__	① __global__ 、__device__、__host__ ② __shared__	新增__simt_vf_ 、__simt_vf_、__gm__ 等修饰符
核函数定义	SIMD/SIMT 混合: ● __simt_vf__ __aicore__ launch_bounds(maxThreadsPerBlock) inline void simtVectorFunction(__ubuf__ float* input, …) ● 混合Kernel函数参考__global__ void myKernel(__gm__ float*,…)	__global__ __launch_bounds(maxThreadsPerBlock, minBlocks) void myKernel(float* input, float*output, int size);	调用方式与纯SIMT不同，分两次调度
核函数调用	SIMD/SIMT混合模式: numBlocks 和 threadsPerBlock 同时支持int或dim3 类型; ● blockDim通过kernel_name<<< numBlocks, dynUBufSize, stream>>>(args...)传入 ● simt函数调用时通过VF_CALL传递参数:asc_vf_call(threadsPerBlock, ...)	numBlocks 和 threadsPerBlock 同时支持int或dim3 类型; kernel_name<<<numBlocks, threadsPerBlock, localMemSize, stream>>>(args...)	
Memory Fence	__aicore__ inline void asc_threadfence()	__threadfence() 、__threadfence_block()、__threadfence_system	函数命名与纯SIMT的函数一致，但函数参数可能需要新增额外修饰符，如__gm__、__ubuf__
同步函数	__aicore__ inline void asc_syncthreads()	__syncthreads()、__syncthreads_count、__syncthreads_and、__syncwarp	
Warp Vote	int32_t asc_all(int32_t predicate) 、asc_any(int32_t predicate)、asc_activemask()	int __all(int predicate); // 9.0之前 int __all_sync(unsigned mask, int predicate); // 9.0新增	

Warp Shuffle	__aicore__ inline int32_t asc_shfl(int32_t var, int32_t srcLane, int32_t width = warpSize)、asc_shfl_up、asc_shfl_down、asc_shfl_xor	int __shfl(T var, int srcLane, int width=warpSize); // 9.0之前
		int __shfl_sync(unsigned mask, int var, int srcLane, int width=warpSize);
		__shf_up_sync、__shfl_down_sync、__shfl_xor_sync	

Warp Reduce	__aicore__ inline unsigned asc_reduce_add(unsigned val) 、asc_reduce_max、asc_reduce_min	unsigned __reduce_add_sync(unsigned mask, unsigned value) 、_reduce_min_sync、_reduce_max_sync	
原子函数	__aicore__ inline int_t asc_atomic_add(__gm__ int *address, int val) 、asc_atomic_sub、asc_atomic_exch等_	int atomicAdd(int* address, int val)、atomicSub、atomicExch等	
Math函数	__aicore__ inline float asinf(float x)、__aicore__ inline float atanf(float x) 等	asinf(x) 、atanf(x) 等	
Launch Bound	__launch_bounds(maxThreadsPerBlock)	__launch_bounds(maxThreadsPerBlock, minBlocks)， minBlocks可选	

Ascend C SIMD+SIMT混合编程实践

SIMT Kernel示例

```
template <typename T>
__simt_vf__ inline void SimtGather(
__ubuf__ T* dst, __ubuf__ T* idx, __ubuf__ T* val, uint32_t len)
{
    int tid = blockDim.x * blockIdx.x + threadIdx.x;

    if (idx < len) {
        dst[idx] =val[tid[idx]];
    }
}
```

SIMD Kernel示例

```
__simd_vf__ inline void SimdAdd(__ubuf__ half* dstPtr, __ubuf__
  half * src0Ptr, __ubuf__ half * src1Ptr, uint32_t len ) {
    constexpr uint16_t oneRepeatSize = asc_get_vf_len() / sizeof(half);
    uint16_t repeatTimes = ceil(len, oneRepeatSize);

    vector_half dstReg0, srcReg0, srcReg1;
    vector_bool maskReg;
    for (uint16_t i = 0; i < repeatTimes; i++) {
        maskReg = asc_update_mask_b16(len);
        asc_load(srcReg0, src0Ptr + i * oneRepeatSize);
        asc_load(srcReg1, src1Ptr + i * oneRepeatSize);
        asc_add(dstReg0, srcReg0, srcReg1, maskReg);
        asc_store(dstPtr + i * oneRepeatSize, dstReg0, maskReg);
    }
}
```

SIMT和SIMD混合编程示例

```
template<typename T>
extern "C" __global__ __aicore__ inline void MixKernel(__gm__ T* x, __gm__ T *  y, __gm__ T * z, uint32_t len)
{
    // Allocate UB Memory
    __ubuf__ T dst0Local[512], src0Local [512], src1Local[512];

    ...
    // Use DataCopy Copy data from GM to LocalTensor
    asc_copy_gm2ub(src0Local, gmTensor, len);
   ...
    dim3 blockDim {32, 1, 1};
    // Execute SIMT Vector Function(VF) -> Use SIMT to gather data
    asc_vf_call<SimtGather<T>>(blockDim, dst0Local, src0Local, src1Local, len);

    // Execute SIMD Vector Function(VF) -> Just for demo, it can be Softmax or other ops
    asc_vf_call<SimdAdd>(dst0Local, src0Local, src1Local, len);
}
```

SIMD & SIMT混合示例:

① 同一个算子同时可包含SIMD和SIMT，充分发挥各自的优势

② 通过VF 函数来区分SIMD和SIMT

Ascend C SIMT算子优化实践

04

优化实践：概述

	优化项	
1	SIMD/SIMT的选择	适用场景
2	任务划分	gridDim配置
3	访存优化	访存合并
		Bank冲突
4	指令优化	整数快速除法

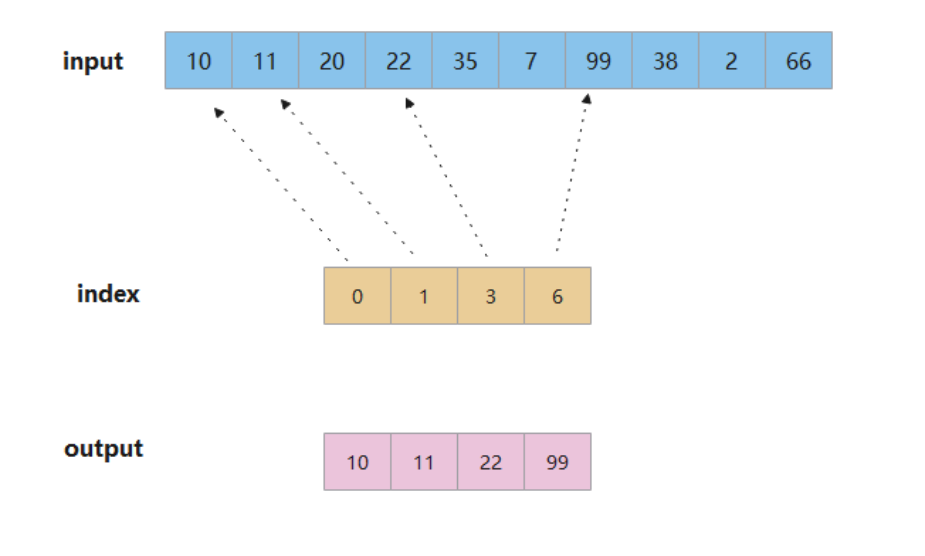
优化实践：SIMT与SIMD的选择

SIMD编程：适用于计算密集型场景，如矩阵计算、密集连续的矢量计算等。
SIMT编程：适用于离散访问和复杂控制逻辑等场景，能够简化离散访问数据、不规则控制流的开发。

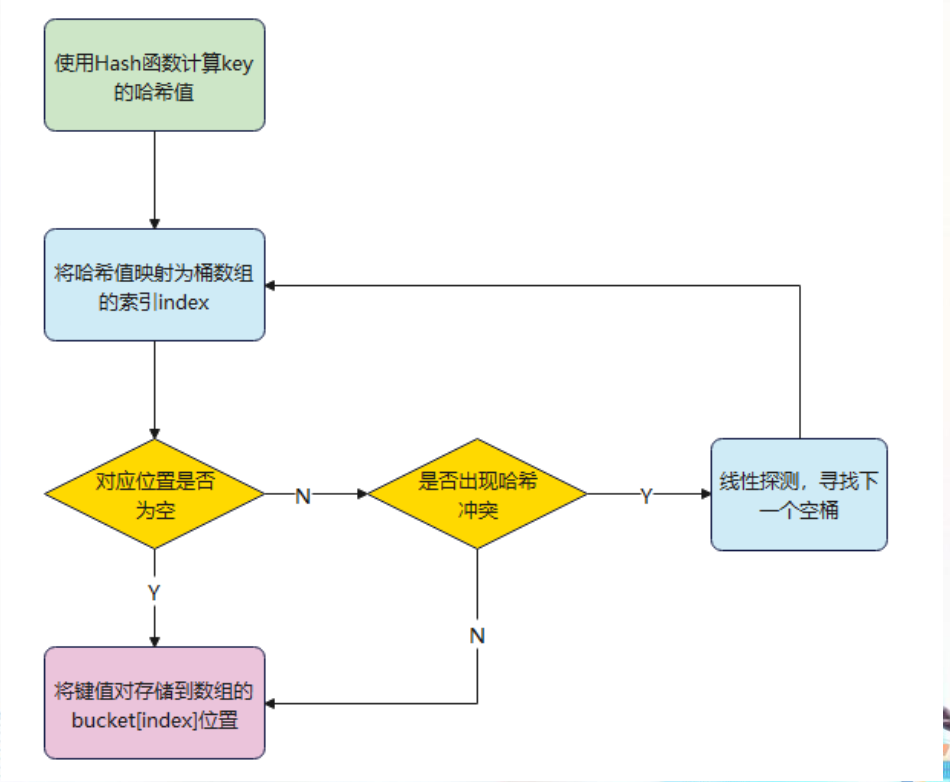
在Ascend950上，SIMT作为SIMD的扩展，补充并增强了芯片的适用场景。典型SIMT场景：

gather算子(离散访存)

$$output_i = input_{index_i}$$



insert_hash_table算子(复杂分支处理)

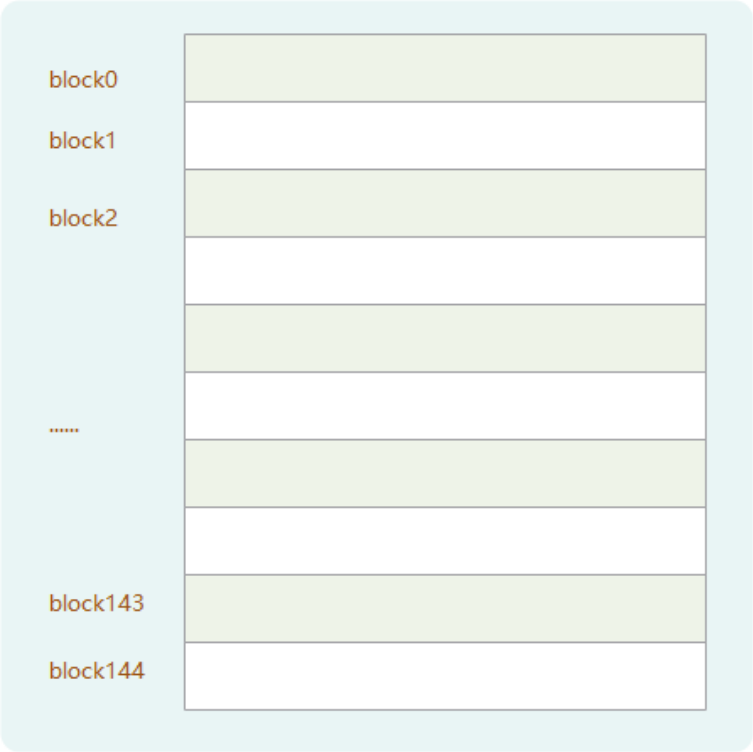


优化实践：任务划分(gridDim配置)

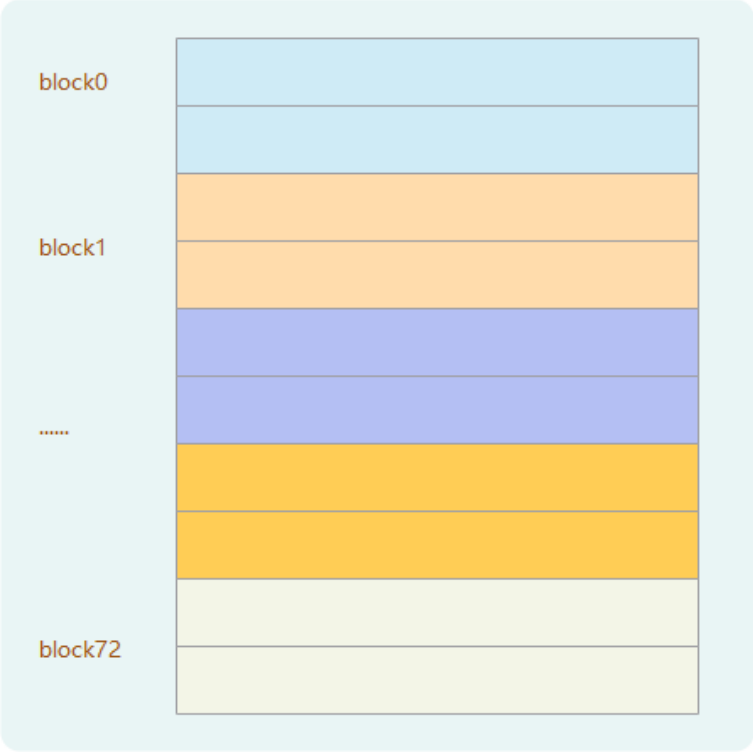
如前文硬件架构描述：
在Ascend 950上，同一时刻一个AIV只能执行一个SIMT Vector Function。
意味着算子gridDim配置超出vector物理核数量时，会带来blcock切换的开销。

如下所示： 假设输入计算数据shape为[144, N]，存在以下两种切分方式

切分方式1：逐行切分，逻辑核数超过物理核，gridDim为144



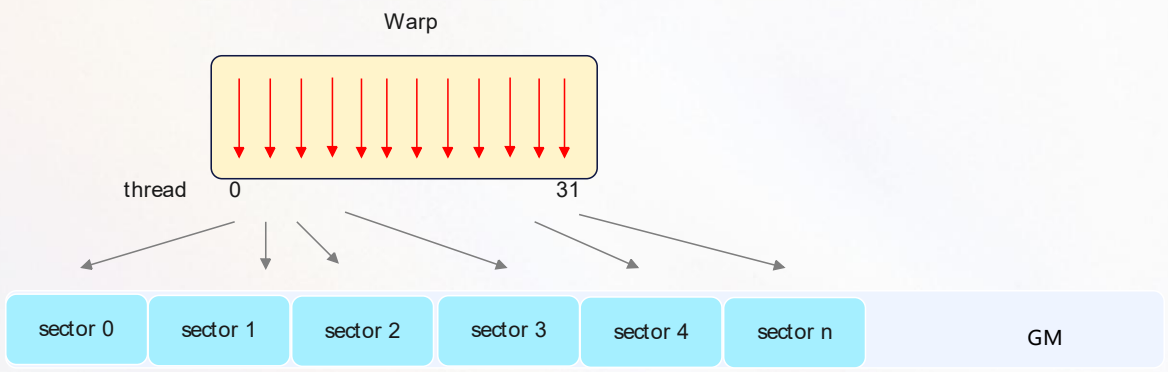
切分方式2：按照物理核数切分，每个核计算2行，gridDim为72



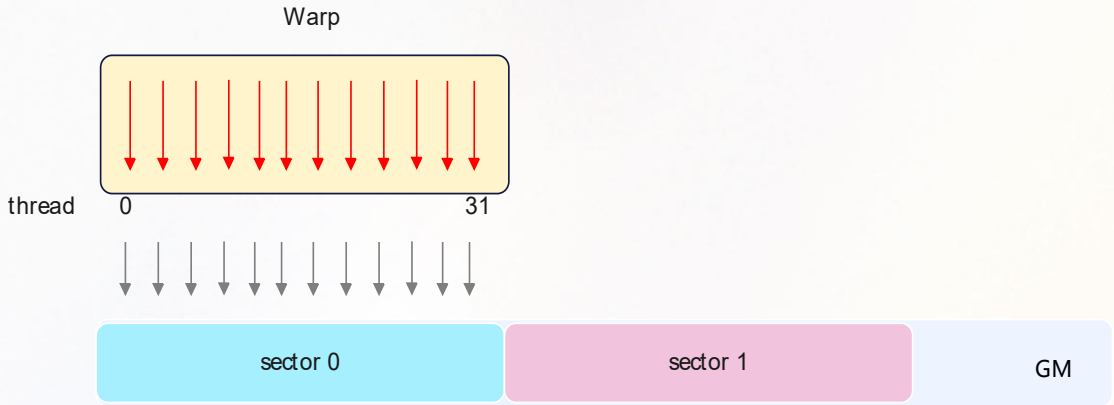
性能优选切分方式2:
减少逻辑核切换开销

优化实践：访存优化(访存合并)

同一个Warp内的线程对全局内存的访问请求，如果落在同一个sector范围内可以合并为同一个内存请求



warp内线程落在n个sector中



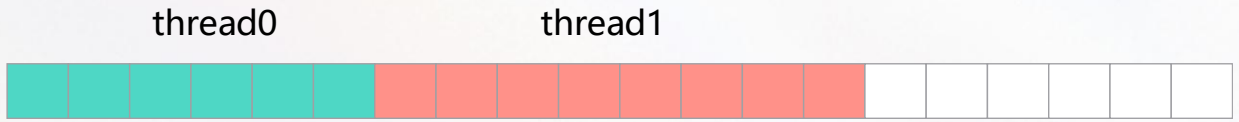
warp内线程落在同1个sector中

最优情况是warp内所有线程落在同一个sector中，此时带宽使用率最高

优化实践：访存优化(访存合并)

下面以一个向量加为例，演示合并访存的技巧。计算过程为Z = X + Y

- 切分方式1：按照thread切分数据，每个thread计算连续的8个元素，每个warp的访存数据不连续



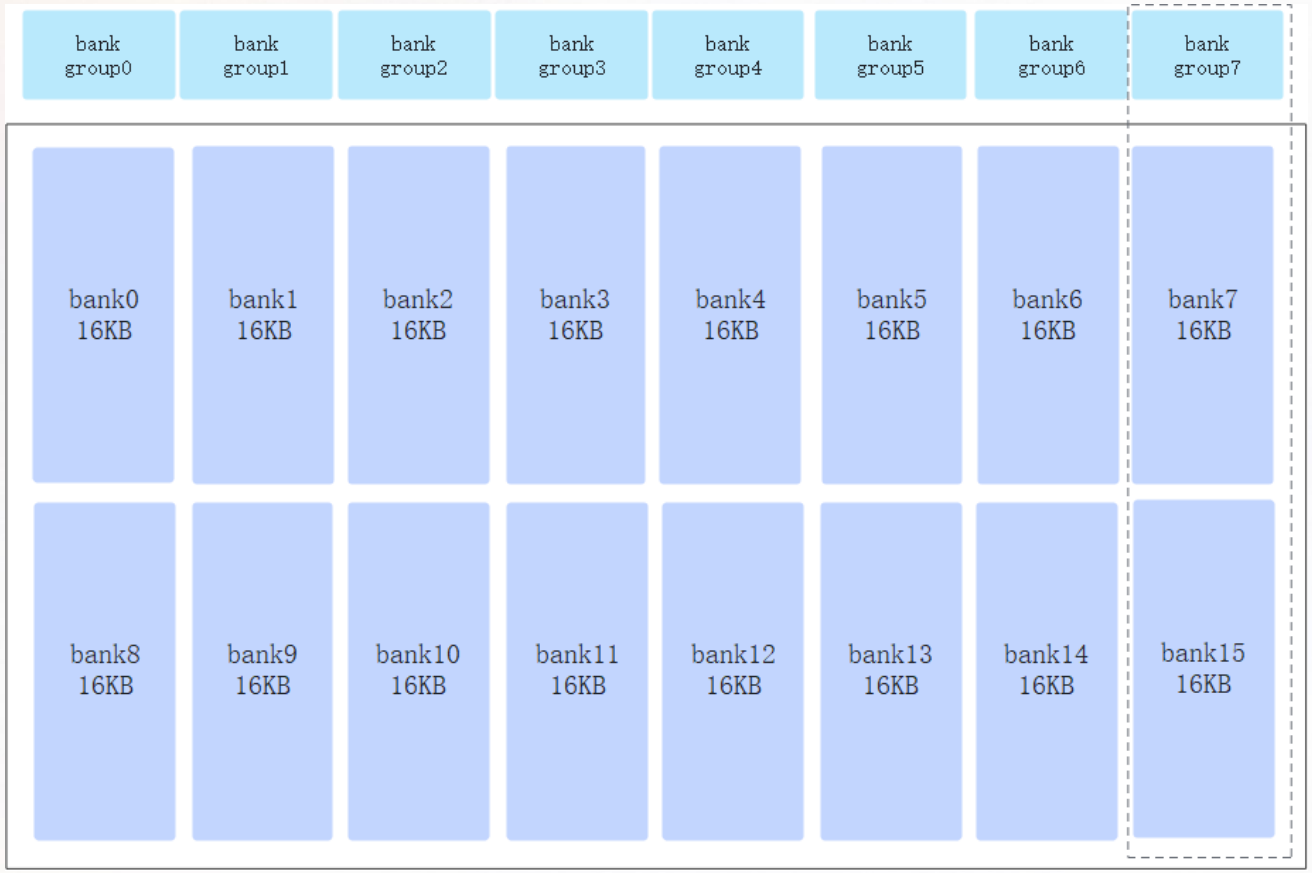

```
int32_t tid = blockIdx.x * blockDim.x + threadIdx.x;
for (int k = 0; k < POINTS_PER_THREAD; k++) {
    int32_t idx = tid * POINTS_PER_THREAD + k;
    if (idx < total_length) {
        z[idx] = x[idx] + y[idx];
    }
}
```

- 切分方式2：按照block切分数据，确保每个warp的访问数据连续

```
int32_t start_id = blockIdx.x * POINTS_PER_BLOCK + threadIdx.x
for (int k = 0; k < POINTS_PER_THREAD; k++) {
   int32_t idx = start_id + k * blockDim.x;
   if (idx < total_length) {
      z[idx] = x[idx] + y[idx];
   }
}
```

优化实践：访存优化(Bank冲突)

在Ascend 950上，UB总大小为256KB，包含8个bank_group，每个Bank大小为16KB



Bank冲突产生的原因：

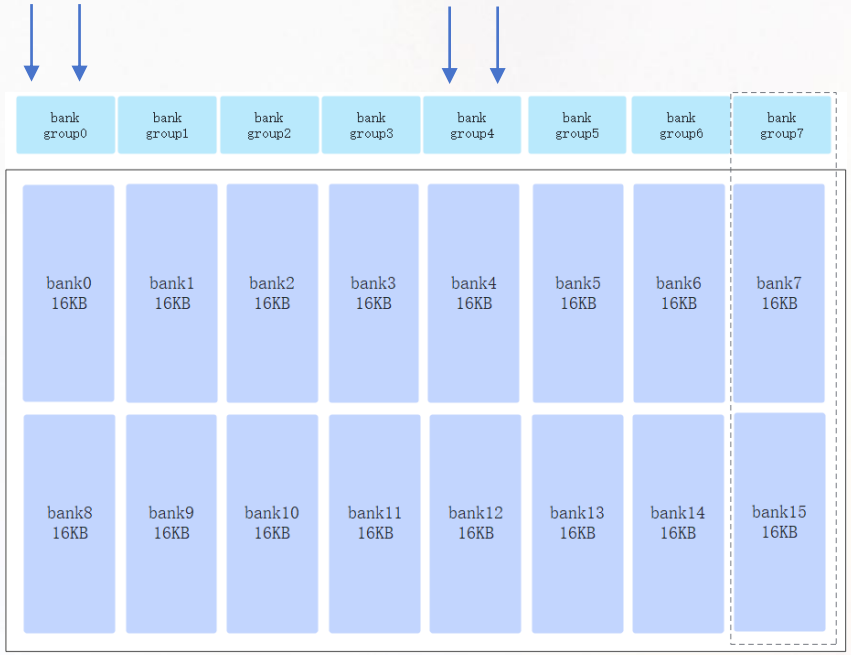
- 读写冲突：读操作和写操作同时尝试访问同一个bank。
- 写写冲突：多个写操作同时尝试访问同一个bank group
- 读读冲突：两个读操作同时尝试访问同一个bank，或者两个以上读操作同时尝试访问同一个bank group

优化实践：访存优化（Bank冲突）

以典型的transpose算子为例

```
__global__ void transpose_coalesced_kernel(float *output, const float *input, int width, int height)
{
    constexpr int32_t TILE_DIM = 32
    // Coalesced global-memory transpose, but shared-memory readback still has bank conflicts.
    __ubuf__ float tile[TILE_DIM][TILE_DIM];

    // Read tile in row-major order from input.
    int x_index = blockIdx.x * TILE_DIM + threadIdx.x;
    int y_index = blockIdx.y * TILE_DIM + threadIdx.y;
    int index_in = x_index + y_index * width;

    // Swap block coordinates to write transposed tile to output.
    x_index = blockIdx.y * TILE_DIM + threadIdx.x;
    y_index = blockIdx.x * TILE_DIM + threadIdx.y;
    int index_out = x_index + y_index * height;

    // Global -> shared.
    for (int i = 0; i < TILE_DIM; i += BLOCK_ROWS) {
        tile[threadIdx.y + i][threadIdx.x] = input[index_in + i * width];
    }

    asc_syncthreads();

    // Shared (transposed access) -> global.
    for (int i = 0; i < TILE_DIM; i += BLOCK_ROWS) {
        output[index_out + i * height] = tile[threadIdx.x][threadIdx.y + i];
    }
}
```

以左侧代码为例，同一个warp内的32个请求，集中在BankGroup0 以及BankGroup4内



解决方法：

1.UB Tile增加一列，大小变为float tile[32][33]；可以缓解Bank冲突到原来的1/4

2.结合应用场景，部分算子中可以考虑使用SIMD+SIMT协同，使用MTE3搬出，规避Bank冲突

优化实践：指令优化(整数快除)

直接使用整数除法时，除法计算会被编译为几十条指令，性能较差。因此在SIMT中如果遇到一个整数向量除以同一个整数时，可以使用快速算法实现。

- 获取magic和shift值

$$shift = \lceil log_2 divisor \rceil$$

$$magic = \lfloor 2^N * (2^{shift} - d) / d \rfloor + 1$$

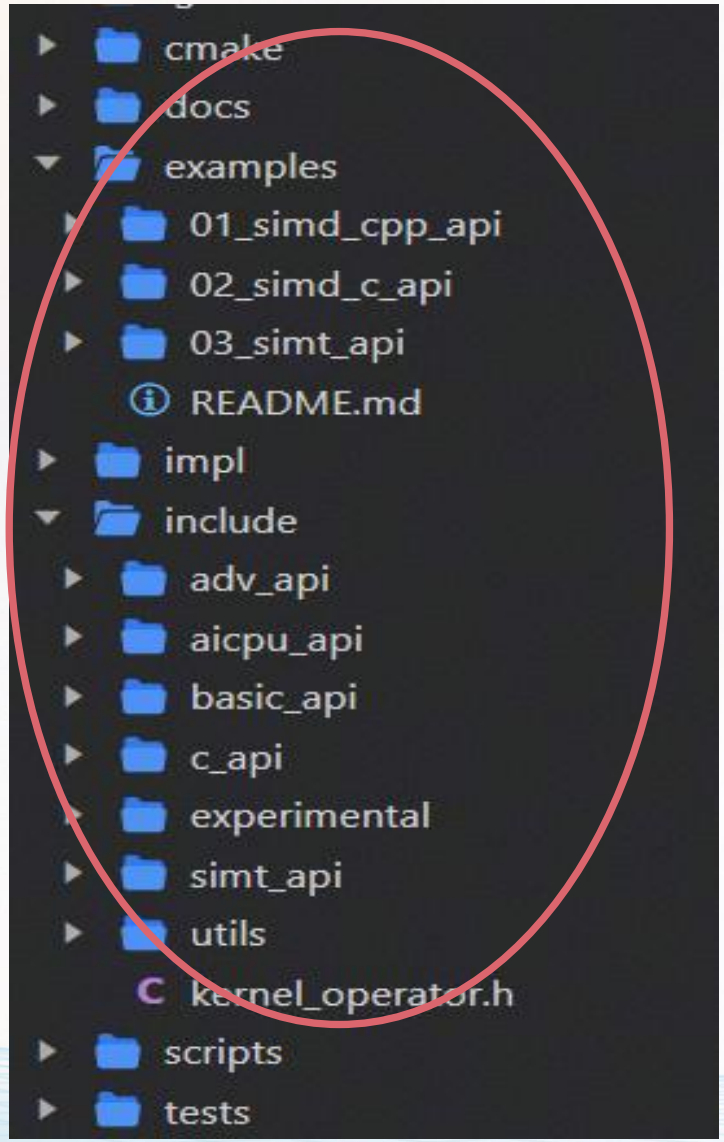
- 根据被除数/magic/shift值获取计算结果

$$quotient = \left(\frac{dividend * magic}{2^N} + dividend \right) \gg shift$$

Ascend C 开源仓介绍

05

Ascend C开源仓介绍



- **docs**：api的介绍以及Ascend C编程指南、入门教程等资料。
- **examples：** 算子开发样例。
- **include：** Ascend C对外头文件：包含simt & simd全量能力。
  c_api： 纯C风格的接口，符合C语言算子开发习惯，提供完备能力
  basic_api：Ascend C基础API，基础API实现对硬件能力的抽象，开放芯片的能力
  simt_api：AscendC的simt api，提供simt的完备能力
  adv_api：高阶API，基于单核对常见算法进行抽象和封装,提高算子开发效率
- **impl：** AscendC接口内部实现。

Ascend C Q2会更新大量SIMT相关样例，并新增协作组等新特性。

欢迎大家使用及发表issue，代码仓链接：
https://gitcode.com/cann/asc-devkit

Thanks!

访问CANN开源社区

关注昇腾CANN公众号